

Pandas DataFrame Cheat Sheet

Comprehensive Modern Handbook & Quick Reference Guide for Data Science in Python

PANDAS V2.X READY

PYTHON 3.9+

COMPLETE REFERENCE

TABLE OF CONTENTS

1. Introduction to Pandas	10. Working with Strings
2. Reading Files	11. Date and Time Functions
3. Viewing Data	12. Exporting Data
4. Selecting Data	13. Useful DataFrame Attributes
5. Data Cleaning	14. Advanced Operations
6. Sorting and Ranking	15. Best Practices & Optimization
7. Statistics	16. Pandas Workflow Diagram
8. Grouping & Aggregation	17. Quick-Reference Cheat Sheet
9. Combining Data	

1 Introduction to Pandas

What is Pandas? Pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation tool built on top of Python and NumPy. Its primary data structure is the **DataFrame**—a 2D labeled tabular structure with rows and columns.

Why use Pandas? It simplifies reading/writing data from various formats, handling missing data, filtering, reshaping, aggregating, and joining datasets with high performance.

Installation:

```
pip install pandas numpy openpyxl pyarrow
```

Standard Import:

```
import pandas as pd
import numpy as np
```

2 Reading Files

`pd.read_csv(filepath_or_buffer, sep=',', header='infer', index_col=None, ...)`

Reads a comma-separated values (CSV) file or delimited text file into a DataFrame.

KEY PARAMETERS

filepath, sep, header, index_col, usecols, dtype, parse_dates

DATA INGESTION STRUCTURE (ILLUSTRATION)

CSV Text → Data Matrix [Rows x Cols]

RETURN VALUE

DataFrame or TextFileReader

```
# Example: Load CSV with custom separator and date parsing
df = pd.read_csv('data.csv', sep=',', usecols=['id', 'date', 'sales'], parse_dates=['date'])
```

	date	sales
id		
101	2023-01-01	250.5
102	2023-01-02	180.0

Tip: Use `usecols` to load only specific columns and save substantial memory on large files.

Common Mistake: Forgetting to pass `parse_dates` leaves date columns as string objects.

`pd.read_excel(io, sheet_name=0, header=0, index_col=None, engine=None)`

Reads an Excel file (.xlsx, .xls) into a Pandas DataFrame.

PARAMETERS

io, sheet_name (str/int/list), engine ('openpyxl')

RETURNS

DataFrame or dict of DataFrames

```
df = pd.read_excel('report.xlsx', sheet_name='Sales2023', engine='openpyxl')
```

Tip: Pass `sheet_name=None` to load all sheets as a dictionary of DataFrames.

`pd.read_json(path_or_buf, orient=None, lines=False)`

Converts a JSON string or JSON file into a Pandas object.

PARAMETERS

path_or_buf, orient ('split'/'records'/'index'), lines (bool)

RETURNS

DataFrame or Series

```
# Load JSON Lines format
df = pd.read_json('data.jsonl', lines=True)
```

Tip: Set `lines=True` when reading JSON Lines (newline-delimited JSON) format.

```
pd.read_parquet(path, engine='auto', columns=None)
```

Loads a Parquet object, returning a DataFrame. High performance columnar binary format.

```
df = pd.read_parquet('data.parquet', columns=['user_id', 'score'])
```

Tip: Parquet provides extreme compression and significantly faster load speeds compared to CSV.

```
pd.read_html(io, match='.+', flavor=None)
```

Read HTML tables into a list of DataFrame objects using BeautifulSoup/lxml under the hood.

```
tables = pd.read_html('https://en.wikipedia.org/wiki/List_of_countries')  
df = tables[0] # First table on page
```

```
pd.read_sql(sql, con, index_col=None, params=None)
```

Read SQL query or database table into a DataFrame using SQLAlchemy engine connection.

```
from sqlalchemy import create_engine  
engine = create_engine('sqlite:///mydb.db')  
df = pd.read_sql('SELECT * FROM users WHERE age > 21', con=engine)
```

```
pd.read_pickle(filepath_or_buffer, compression='infer')
```

Load pickled pandas object (Python binary object serialization) from file.

```
df = pd.read_pickle('processed_data.pkl')
```

Tip: Preserves exact Pandas dtypes (including categorical and datetimes) perfectly.

3 Viewing Data

```
df.head(n=5) / df.tail(n=5) / df.sample(n=1, frac=None)
```

Inspect rows of the DataFrame. head returns top n rows, tail returns bottom n rows, and sample returns random rows.

```
df.head(3) # First 3 rows  
df.sample(n=2, random_state=42) # 2 Random rows
```

df.info(verbose=None, show_counts=None)

Prints a concise summary of a DataFrame including index dtype, column dtypes, non-null counts, and memory usage.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   age     98 non-null     float64
1   city    100 non-null    object
2   salary  95 non-null     float64
dtypes: float64(2), object(1)
memory usage: 2.5+ KB
```

df.describe(percentiles=None, include=None, exclude=None)

Generates descriptive statistics summarizing central tendency, dispersion, and shape of a dataset's distribution.

```
df.describe()
```

	age	salary
count	98.000000	95.000000
mean	34.500000	65000.000000
std	10.200000	12000.000000
min	18.000000	30000.000000
50%	33.000000	62000.000000
max	65.000000	98000.000000

Structural Attributes: df.shape, df.columns, df.index, df.dtypes

Core properties to inspect dimensions and column names.

```
print(df.shape)    # (100, 3) -> tuple (rows, cols)
print(df.columns)  # Index(['age', 'city', 'salary'], dtype='object')
print(df.dtypes)   # Returns Series with column data types
```

4 Selecting Data

Column Selection: Single & Multiple

Select columns using bracket notation or attribute access. Pass a list of names for multiple columns.

```
# Single column (returns Series)
age_series = df['age']

# Multiple columns (returns DataFrame)
subset = df[['age', 'salary']]
```

`df.loc[row_indexer, column_indexer]`

Label-based selection. Access rows/columns by explicit labels or a boolean array.

```
# Select rows with index 'a' to 'c' and specific columns
df.loc['a':'c', ['age', 'salary']]

# Label conditional selection
df.loc[df['age'] > 30, 'salary']
```

Tip: Unlike standard Python slices, `.loc` end boundary slicing IS INCLUSIVE!

`df.iloc[row_indexer, column_indexer]`

Integer position-based selection. Access rows/columns by 0-based integer positions.

```
# First 5 rows, first 2 columns
df.iloc[0:5, 0:2]
```

Common Mistake: `.iloc` uses standard Python half-open interval slicing (endpoint is EXCLUDED).

Boolean Indexing & Query Method

Filter rows using logical conditions (& for AND, | for OR, ~ for NOT) or string queries.

```
# Boolean indexing
high_earners = df[(df['age'] > 30) & (df['salary'] >= 70000)]

# Using .query() syntax (cleaner for complex filters)
min_sal = 70000
high_earners = df.query('age > 30 and salary >= @min_sal')
```

5 Data Cleaning

`df.isnull() / df.isna() & df.notnull()`

Detect missing values. Returns a boolean mask indicating missing values (NaN/None).

```
# Count missing values per column
df.isnull().sum()
```

```
age      2
city     0
salary   5
dtype: int64
```

df.fillna(value=None, method=None, inplace=False)

Fill missing (NaN) values using a specified value, mean/median, or propagation method.

```
# Fill missing salary with mean salary
df['salary'] = df['salary'].fillna(df['salary'].mean())
```

df.dropna(axis=0, how='any', thresh=None, subset=None)

Remove missing values along specified axis.

```
# Drop rows where 'age' or 'salary' is missing
df_clean = df.dropna(subset=['age', 'salary'])
```

df.replace(to_replace=None, value=None) & df.astype(dtype)

Replace values or cast series to a specified data type.

```
# Replace invalid strings
df['city'] = df['city'].replace({'NY': 'New York'})

# Cast data types
df['age'] = df['age'].astype(int)
```

df.duplicated(subset=None, keep='first') & df.drop_duplicates()

Identify and drop duplicate rows from DataFrame.

```
df_unique = df.drop_duplicates(subset=['user_id'], keep='last')
```

6 Sorting and Ranking

df.sort_values(by, ascending=True, inplace=False)

Sort DataFrame by values along either axis.

```
df_sorted = df.sort_values(by=['city', 'salary'], ascending=[True, False])
```

df.sort_index(axis=0, ascending=True) & df.rank(method='average')

Sort by row index or compute numerical ranks (1 through n) along axis.

```
df['salary_rank'] = df['salary'].rank(ascending=False, method='dense')
```

```
df.nlargest(n, columns) / df.nsmallest(n, columns)
```

Get top/bottom n rows ordered by specified columns. Fast alternative to full sort.

```
top_3 = df.nlargest(3, 'salary')
```

7 Statistics

Summary Statistical Functions

Pandas provides easy aggregate methods across DataFrame columns or rows (axis=0 for columns, axis=1 for rows).

Method	Description	Example Syntax
mean()	Arithmetic mean of values	df['salary'].mean()
median()	50th percentile / median value	df['age'].median()
mode()	Most frequent value(s)	df['city'].mode()[0]
min() / max()	Minimum / Maximum value	df['salary'].max()
std() / var()	Standard deviation / Variance	df['salary'].std()
count()	Number of non-null observations	df['age'].count()
sum()	Sum of values	df['sales'].sum()
corr()	Pairwise correlation of columns	df.corr(numeric_only=True)

```
# Pairwise Pearson Correlation Matrix
correlations = df[['age', 'salary']].corr()
```

8 Grouping and Aggregation

```
df.groupby(by, as_index=True).agg()
```

Group DataFrame using a mapper or by Series of columns. Apply aggregate functions on groups.

```
# Group by city and calculate stats
grouped = df.groupby('city').agg(
    avg_salary=('salary', 'mean'),
    max_age=('age', 'max'),
    count=('salary', 'count')
)
```

	avg_salary	max_age	count
city			
Chicago	70000.0	45	2
New York	85000.0	50	3

`df.groupby().transform(func)`

Returns an object that is indexed the same (same length) as the original DataFrame. Great for group-level feature engineering.

```
# Calculate mean salary of person's city and append as column
df['city_avg_salary'] = df.groupby('city')['salary'].transform('mean')
```

`df.value_counts(normalize=False) & pd.pivot_table()`

Count unique value combinations or create a spreadsheet-style pivot table.

```
# Frequency table
counts = df['city'].value_counts(normalize=True)

# Pivot Table
pivot = pd.pivot_table(df, values='salary', index='city', columns='dept', aggfunc='mean')
```

9 Combining Data

`pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None)`

Merge DataFrame objects with a database-style join (inner, outer, left, right).

```
merged = pd.merge(df1, df2, on='user_id', how='left')
```

`pd.concat(objs, axis=0, ignore_index=False)`

Concatenate pandas objects along a particular axis (axis=0 for stacking rows, axis=1 for joining columns).

```
# Stack DataFrames vertically
combined = pd.concat([df1, df2], axis=0, ignore_index=True)
```

`df.join(other, how='left') & Legacy Note`

Join columns of another DataFrame on index.

Legacy Note: `df.append()` has been deprecated since Pandas 1.4+ and removed in 2.0. Use `pd.concat()` instead.

10 Working with Strings

Vectorized String Accessor: `Series.str`

Apply string operations across Series items efficiently.

```
# String transformations
df['name_upper'] = df['name'].str.upper()
df['has_tech'] = df['job'].str.contains('Tech', case=False, na=False)
df['clean_code'] = df['code'].str.replace('-', '')
df['first_name'] = df['full_name'].str.split(' ').str[0]
```

11 Date and Time Functions

`pd.to_datetime()` & Datetime Accessor: `Series.dt`

Convert arguments to datetime and extract date properties (year, month, day, weekday, etc.).

```
df['date'] = pd.to_datetime(df['date_string'], format='%Y-%m-%d')

# Extracting Datetime properties
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['weekday'] = df['date'].dt.weekday # Monday=0, Sunday=6
```

12 Exporting Data

DataFrame Export Methods

Write DataFrame content to various output files or storage formats.

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', sheet_name='Results', index=False)
df.to_json('output.json', orient='records')
df.to_html('table.html', index=False)
df.to_pickle('data.pkl')
```

Tip: Always set `index=False` unless you explicitly need the row labels in your saved file.

13 Useful DataFrame Attributes

Attribute	Type	Description & Example
shape	tuple	Returns (n_rows, n_cols) tuple → (100, 5)
size	int	Total elements count (rows x cols) → 500
ndim	int	Number of dimensions (2 for DataFrame)
values	ndarray	Numpy 2D array representation of DataFrame values
columns	Index	Column labels → Index(['a', 'b'])
index	Index	Row labels / Index object
empty	bool	True if DataFrame has no data (0 rows or 0 cols)

14 Advanced Operations

df.apply(func, axis=0) & Series.map(arg)

Apply function along axis or map values of a Series using dynamic dictionary lookup / lambda function.

```
# apply across columns or rows
df['double_age'] = df['age'].apply(lambda x: x * 2)

# map value substitution
gender_map = {'M': 0, 'F': 1}
df['gender_code'] = df['gender'].map(gender_map)
```

df.pipe(func, *args, **kwargs)

Apply chainable functions that accept a DataFrame as input. Enables clean visual method pipelines.

```
def clean_data(data):
    return data.dropna().reset_index(drop=True)

df_clean = (df
            .pipe(clean_data)
            .assign(adjusted_sal=df['salary']*1.1))
```

df.explode(column) & df.melt(id_vars, value_vars)

Reshape DataFrames: explode unrolls lists into rows; melt unpivots wide format to long format.

```
# Explode list-like elements to individual rows
df_exploded = df.explode('tags')

# Wide to Long Format
df_long = df.melt(id_vars=['id'], value_vars=['Q1', 'Q2'], var_name='Quarter', value_name='Score')
```

15 Best Practices & Memory Optimization

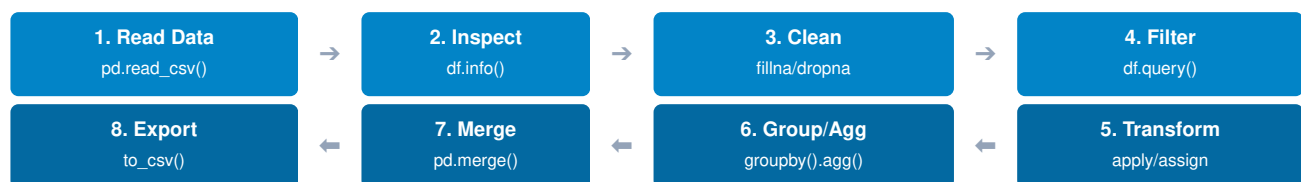
- **Vectorization over Iteration:** Never use for loops or `iterrows()` for computations. Vectorized operations are written in C and run up to 100x faster.
- **Use Categorical Dtypes:** Convert high-cardinality string columns with repetitive text (e.g. state, category) to category dtype to reduce memory usage by up to 80%.

```
# Memory reduction example
df['category'] = df['category'].astype('category')
```

- **Downcast Numeric Types:** Use `pd.to_numeric(df['col'], downcast='integer')` to convert int64 to int32/int8.
- **Method Chaining:** Write clean pipelines using wrapped parentheses and Pandas chaining for better readability.
- **Avoid inplace=True:** In modern Pandas, `inplace=True` offers no performance benefit and hinders operation chaining.

16 Pandas Data Processing Workflow

End-to-End Data Science Pipeline in Pandas



17 Quick-Reference Cheat Sheet Table

A fast lookup table covering core operations, functions, and standard syntax.

Method	Description	Syntax	Example
<code>pd.read_csv</code>	Load CSV data into DataFrame	<code>pd.read_csv(filepath)</code>	<code>df = pd.read_csv('data.csv')</code>
<code>df.head(n)</code>	Return top n rows	<code>df.head(n=5)</code>	<code>df.head(10)</code>
<code>df.info()</code>	Print summary of DataFrame	<code>df.info()</code>	<code>df.info()</code>
<code>df.describe()</code>	Generate summary statistics	<code>df.describe()</code>	<code>df.describe()</code>
<code>df.loc[]</code>	Select by labels	<code>df.loc[row_lbl, col_lbl]</code>	<code>df.loc[0:5, 'age']</code>
<code>df.iloc[]</code>	Select by integer index	<code>df.iloc[row_idx, col_idx]</code>	<code>df.iloc[0:5, 0:2]</code>
<code>df.query()</code>	Filter rows using string expression	<code>df.query(expr)</code>	<code>df.query('age > 25')</code>
<code>df.fillna()</code>	Fill NA/NaN values	<code>df.fillna(val)</code>	<code>df.fillna(0)</code>
<code>df.dropna()</code>	Remove missing values	<code>df.dropna(subset=[...])</code>	<code>df.dropna(subset=['age'])</code>
<code>df.drop_duplicates()</code>	Remove duplicate rows	<code>df.drop_duplicates()</code>	<code>df.drop_duplicates('id')</code>
<code>df.sort_values()</code>	Sort by column values	<code>df.sort_values(by=...)</code>	<code>df.sort_values('salary')</code>
<code>df.groupby()</code>		<code>df.groupby(by)</code>	<code>df.groupby('city').mean()</code>

Method	Description	Syntax	Example
	Group rows by column values		
<code>pd.merge()</code>	Database-style merge/join	<code>pd.merge(df1, df2, on=...)</code>	<code>pd.merge(d1, d2, on='id')</code>
<code>pd.concat()</code>	Concatenate objects along axis	<code>pd.concat([d1, d2], axis=0)</code>	<code>pd.concat([d1, d2])</code>
<code>Series.str.contains()</code>	Test if pattern is in string	<code>s.str.contains(pat)</code>	<code>df['name'].str.contains('John')</code>
<code>pd.to_datetime()</code>	Convert argument to datetime	<code>pd.to_datetime(arg)</code>	<code>pd.to_datetime(df['date'])</code>
<code>df.apply()</code>	Apply function along axis	<code>df.apply(func, axis=0)</code>	<code>df['x'].apply(lambda x: x*2)</code>
<code>df.melt()</code>	Unpivot wide to long format	<code>df.melt(id_vars, ...)</code>	<code>df.melt(id_vars=['id'])</code>
<code>df.to_csv()</code>	Export DataFrame to CSV	<code>df.to_csv(filename)</code>	<code>df.to_csv('out.csv', index=False)</code>